Apuntes de Git, GitHub Pages y GitHub Actions

2º DAW · Despliegue de aplicaciones web
Guía de examen con código listo para copiar y pegar

ATAJOS. Pulsa en cualquier línea del índice para saltar a esa sección. Atajos: Examen resuelto paso a paso · Preguntas de razonamiento · Errores típicos · Chuleta final.

CÓMO USAR. Los bloques de código usan las variables `$USUARI` (tu usuario de GitHub) y `$NOM` (tu nombre y apellidos). Defínelas una vez al abrir la terminal (apartado 1.3) y podrás copiar todo sin editar. Cambia `Nom Cognom` y `correu@gva.es` por tus datos reales.

Índice

1. Preparación del entorno (antes de empezar) . . . 4

1.1 Identidad de Git . . . 4
1.2 Token de GitHub y autenticación con gh . . . 4
1.3 Variables de comodidad (opcional pero muy útil) . . . 4

2. Conceptos clave . . . 5

3. Comandos básicos del día a día . . . 5

3.1 Crear o clonar un repositorio . . . 5
3.2 Ver el estado . . . 5
3.3 Añadir y confirmar (add + commit) . . . 6
3.4 Ver el historial . . . 6
3.5 Mover, borrar y restaurar ficheros . . . 6

4. Repositorios remotos y GitHub . . . 7

4.1 Crear el repo en GitHub (web o línea de comandos) . . . 7
4.2 Enlazar el repo local con el remoto . . . 7
4.3 Subir el trabajo: git push . . . 7
4.4 Bajar el trabajo: git fetch y git pull . . . 7
4.5 Receta completa: de cero a GitHub . . . 8

5. Ramas y fusiones (branch, checkout/switch, merge) . . . 8

5.1 Crear, listar, cambiar y borrar ramas . . . 8
5.2 Fusionar (merge) . . . 9
5.3 Flujo completo con rama de funcionalidad . . . 9
5.4 Resolver un conflicto de merge . . . 9
5.5 Metodologías de ramas (Git Flow) . . . 9

6. Etiquetas (tags) . . . 10

7. .gitignore . . . 10

8. Deshacer cosas . . . 11

9. Colaborar: fork y pull request . . . 12

9.1 Fork . . . 12

9.2 Pull request (PR) . . . 12

10. Buenas prácticas con Git . . . 13

10.1 Mensajes de commit . . . 13

10.2 Reglas de oro . . . 13

11. GitHub Pages . . . 13

11.1 Activarlo desde la web . . . 13

11.2 Activarlo desde la terminal . . . 13

11.3 Receta: repo <usuari>.github.io con web . . . 14

12. CI/CD y GitHub Actions: conceptos . . . 14

12.1 Qué es CI/CD . . . 14

12.2 Componentes de GitHub Actions . . . 15

12.3 Runners . . . 15

13. Workflows: sintaxis y ejemplos listos para copiar . . . 15

13.1 Primer workflow y anatomía . . . 15

13.2 Crear y subir el workflow . . . 16

13.3 Eventos (on) . . . 16

13.4 Ejemplo del examen: tests unitarios en Windows, Linux y Mac . . . 17

13.5 Lo mismo con matrix (más corto) . . . 18

13.6 Dependencias entre jobs (needs) y condiciones (if) . . . 18

13.7 Variables de entorno, run multilínea y contextos . . . 19

13.8 Otros ejemplos útiles . . . 19

14. Custom GitHub Actions (acciones propias) . . . 20

14.1 Tipos . . . 20

14.2 El fichero action.yml . . . 21

14.3 Ejemplo JavaScript paso a paso . . . 21

14.4 Ejemplo Docker paso a paso . . . 22

14.5 Ejemplo composite (pasos compuestos) . . . 23

15. Examen resuelto paso a paso (copiar y pegar) . . . 24

Paso 0. Preparación (no puntúa) . . . 24

Ejercicio 1: Repo (2 puntos) . . . 24

Ejercicio 2: Repo (2 puntos) . . . 25

Ejercicio 3: .gitignore (1 punto) . . . 25

Ejercicio 4: GitHub Pages (2 puntos) . . . 26

Ejercicio 5: GitHub Actions (3 puntos) 26
Comprobación final de todo 27

16. Preguntas cortas de razonamiento (con respuesta) 28

17. Errores típicos y cómo arreglarlos 29

18. Chuleta final de comandos 30

1. Preparación del entorno (antes de empezar)

Esto es lo primero que se hace en la VM del examen. Copia y pega bloque a bloque.

1.1 Identidad de Git

▶ Configurar nombre, correo y rama por defecto main

```
git config --global user.name "Nom Cognom"
git config --global user.email "correu@gva.es"
git config --global init.defaultBranch main
git config --global --list
```

> **TRUCO.** `init.defaultBranch main` hace que `git init` cree la rama `main` directamente (el examen exige que todos los repos estén en `main`).

1.2 Token de GitHub y autenticación con gh

- GitHub → foto de perfil → **Settings** → **Developer settings** → **Personal access tokens** → **Tokens (classic)** → **Generate new token (classic)**.
- Pon un nombre (ej. `examen`), una caducidad y **marca todos los permisos (scopes)**.
- Copia el token (solo se muestra una vez) y guárdalo en un fichero `auth.txt`.

▶ Guardar el token en ~/auth.txt (fuera de cualquier repositorio)

```
cat > ~/auth.txt
# pega el token, pulsa Enter y luego Ctrl+D
```

▶ Iniciar sesión con gh y configurar Git para usar esas credenciales

```
gh auth login --with-token < ~/auth.txt
gh auth setup-git
gh auth status
```

> **OJO. Nunca subas `auth.txt` a un repo.** Si el directorio donde lo guardas es un repositorio (por ejemplo `~/examen`), no uses `git add .`: añade los ficheros uno a uno. Un token en un repo público se detecta y GitHub lo revoca.

1.3 Variables de comodidad (opcional pero muy útil)

Así todos los comandos de estos apuntes se pueden copiar tal cual, sin editar el usuario. Tendrás que ponerlas otra vez si abres una terminal nueva.

▶ Definir usuario de GitHub, nombre completo y primer apellido (minúsculas, sin acentos)

```
USUARI=$(gh api user -q .login)
NOM="Nom Cognom"
COGNOM="cognom"
echo $USUARI
```

2. Conceptos clave

Git guarda el historial de un proyecto como una cadena de **commits** (fotos del proyecto). Un cambio pasa por cuatro zonas:

Zona	Qué es	Comando que mueve el cambio
Working directory	Los ficheros que ves y editas en la carpeta.	`git add` → pasa a staging
Staging area (index)	Lo que irá en el próximo commit.	`git commit` → pasa al repo local
Repositorio local	El historial de commits en tu máquina (carpeta `.git`).	`git push` → pasa al remoto
Repositorio remoto	El repo en GitHub (normalmente llamado `origin`).	`git pull` / `git fetch` → vuelve al local

Término	Significado
Commit	Foto del proyecto con un mensaje y un identificador (hash).
Rama (branch)	Puntero móvil a un commit. Crear una rama no copia nada, solo crea un puntero.
HEAD	Puntero a la rama (o commit) en la que estás trabajando ahora.
origin	Nombre por defecto del remoto. `git clone` lo crea automáticamente.
upstream	Nombre habitual del repo original cuando trabajas con un fork.
Tag (etiqueta)	Marca fija sobre un commit, típica para versiones (v1.0).
Fast-forward	Merge sin commit nuevo: la rama solo avanza el puntero.

Estados de un fichero

- **Untracked**: Git no lo conoce todavía (hace falta `git add`).
- **Modified**: está en Git pero lo has cambiado.
- **Staged**: marcado con `git add` para el próximo commit.
- **Committed**: guardado en el historial.

3. Comandos básicos del día a día

3.1 Crear o clonar un repositorio

▶ Convertir la carpeta actual en repositorio (rama main). Si tu git es antiguo:

```
git init -b main
```

▶ Alternativa para versiones antiguas de Git

```
git init
git branch -M main
```

▶ Clonar un repo (con carpeta de destino opcional) o con gh

```
git clone https://github.com/$USUARI/exercici-1.git
git clone https://github.com/$USUARI/exercici-1.git altre-nom
gh repo clone $USUARI/exercici-1
```

3.2 Ver el estado

▶ status y diff (sin cambios en staging / en staging / local vs remoto / dos commits)

```
git status
git status -s
git diff
git diff --staged
git diff main origin/main
git diff abc1234 def5678
```

`git status -s` muestra `??` (untracked), `M` (modificado), `M ` (staged), `A ` (nuevo en staging).

3.3 Añadir y confirmar (add + commit)

▶ add: un fichero / varios / carpeta / TODO (evitar) / por trozos. commit: normal / add+commit de lo ya seguido

```
git add fitxer.txt
git add index.html Readme.md
git add carpeta/
git add .
git add -p
git commit -m "Missatge del commit"
git commit -am "Missatge"
```

EXAMEN. En el examen te piden cosas como «añade **solo** index.html». Usa siempre `git add nombre_fichero` y comprueba con `git status` antes del commit. `git commit -am` NO añade ficheros nuevos (untracked).

▶ Ejemplo completo: crear un fichero, añadirlo y hacer commit

```
echo "# Respostes — $NOM" > respostes.md
git add respostes.md
git commit -m "Primer commit"
```

▶ Arreglar el ÚLTIMO commit (mensaje mal escrito / se olvidó un fichero con git add antes)

```
git commit --amend -m "Missatge corregit"
git commit --amend --no-edit
```

OJO. `--amend` reescribe el commit. Hazlo solo si **aún no has hecho push**. Si ya lo subiste tendrías que forzar el push; en el examen es mejor un commit nuevo.

3.4 Ver el historial

▶ Variantes de git log y git show

```
git log
git log --oneline
git log -3
git log --oneline -5
git log --oneline --graph --all --decorate
git log --stat
git log -p fitxer.txt
git log --author="Nom"
git log --since="2 days ago"
git show abc1234
git show HEAD
```

▶ Guardar la salida de git log --oneline en un fichero y confirmarlo

```
git log --oneline > log.txt
git add log.txt
git commit -m "Tercer commit"
```

TRUCO. `>` sobrescribe el fichero y `>>` añade al final. Para «añade tu nombre al final del fichero» usa `>>`: `echo "$NOM" >> README.md`.

3.5 Mover, borrar y restaurar ficheros

▶ renombrar / borrar y registrar / dejar de seguir pero conservar / descartar cambios / sacar de staging

```
git mv vell.txt nou.txt
git rm fitxer.txt
git rm --cached fitxer.txt
git restore fitxer.txt
git restore --staged fitxer.txt
```

4. Repositorios remotos y GitHub

4.1 Crear el repo en GitHub (web o línea de comandos)

- **Web**: github.com → **+** → **New repository** → nombre exacto → **Public** → (no marques README ni .gitignore si vas a subir un repo local ya existente) → **Create repository**.
- **Terminal** (más rápido y sin errores de nombre):

▶ Crear un repo público VACÍO en tu cuenta

```
gh repo create exercici-1 --public
```

▶ Crear el repo remoto desde la carpeta actual, enlazar origin y hacer push (necesita ≥1 commit)

```
gh repo create exercici-1 --public --source=. --remote=origin --push
```

▶ Listar tus repos / abrirlo en el navegador / borrarlo (necesita el permiso delete_repo)

```
gh repo list --limit 20
gh repo view $USUARI/exercici-1 --web
gh repo delete $USUARI/exercici-1 --yes
```

4.2 Enlazar el repo local con el remoto

▶ Añadir el remoto origin, listar remotos, ver detalle

```
git remote add origin https://github.com/$USUARI/exercici-1.git
git remote -v
git remote show origin
```

▶ Cambiar la URL de un remoto / renombrarlo / quitarlo

```
git remote set-url origin https://github.com/$USUARI/exercici-1.git
git remote rename origin antic
git remote remove origin
```

> **TRUCO.** Error «remote origin already exists» → ya lo tenías; usa `git remote set-url origin URL`.

4.3 Subir el trabajo: git push

▶ Primer push: -u asocia la rama local con la remota

```
git push -u origin main
```

▶ Después del primer push: normal / explícito / otra rama / todas las etiquetas / borrar rama remota

```
git push
git push origin main
git push origin nom-rama
git push origin --tags
git push origin --delete nom-rama
```

4.4 Bajar el trabajo: git fetch y git pull

Comando	Qué hace
`git fetch origin`	Descarga los cambios del remoto a ramas ocultas (origin/main) pero NO toca tu rama ni tus ficheros.
`git pull`	= `git fetch` + `git merge`: descarga e integra en tu rama actual.
`git pull --rebase`	Descarga y recoloca tus commits encima de los remotos (historial lineal).

▶ Ver qué hay nuevo en remoto antes de integrarlo, y luego integrar

```
git fetch origin
git log --oneline main..origin/main
git diff main origin/main
git merge origin/main
git pull origin main
```

OJO. Haz commit (o stash) de tus cambios **antes** de un `git pull`.

4.5 Receta completa: de cero a GitHub

▶ Carpeta nueva → repo → remoto → primer commit → push

```
mkdir ~/examen && cd ~/examen
git init -b main
git remote add origin https://github.com/$USUARI/exercici-1.git
echo "# Hola" > Readme.md
git add Readme.md
git commit -m "Primer commit"
git push -u origin main
```

▶ Lo que GitHub te enseña al crear un repo vacío (create a new repository on the command line)

```
echo "# ejemploRepo" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/USUARI/ejemploRepo.git
git push -u origin main
```

5. Ramas y fusiones (branch, checkout/switch, merge)

Las ramas permiten desarrollar en paralelo sin tocar `main`. Crear una rama es solo crear un puntero; **crearla no te cambia a ella**: hay que hacer checkout/switch.

5.1 Crear, listar, cambiar y borrar ramas

▶ listar locales / todas / remotas / con su remoto asociado / crear / cambiar / crear+cambiar (2 formas)

```
git branch
git branch -a
git branch -r
git branch -vv
git branch nova-rama
git checkout nova-rama
git checkout -b nova-rama
git switch nova-rama
git switch -c nova-rama
```

▶ borrar rama fusionada / forzar borrado / borrar en remoto / renombrar actual a main / renombrar

```
git branch -d nova-rama
git branch -D nova-rama
git push origin --delete nova-rama
git branch -M main
git branch -m nom-antic nom-nou
```

▶ Traer a local una rama que solo existe en el remoto (tras git fetch)

```
git checkout nom-rama-remota
git switch nom-rama-remota
git checkout -b local origin/remota
```

5.2 Fusionar (merge)

Te colocas en la rama que **recibe** los cambios y fusionas la otra.

▶ Fusionar nova-rama dentro de main y subirlo

```
git switch main
git pull
git merge nova-rama
git push
```

▶ forzar commit de merge aunque pudiera ser fast-forward / cancelar un merge con conflictos

```
git merge --no-ff nova-rama
git merge --abort
```

5.3 Flujo completo con rama de funcionalidad

▶ Crear rama → trabajar → subir → fusionar en main → limpiar

```
git switch -c feature/login
echo "login" > login.txt
git add login.txt
git commit -m "Afegeix login"
git push -u origin feature/login
git switch main
git merge feature/login
git push
git branch -d feature/login
git push origin --delete feature/login
```

5.4 Resolver un conflicto de merge

Si dos ramas cambian la misma línea, Git no sabe cuál elegir y marca el fichero:

▶ Aspecto de un fichero en conflicto

```
<<<<<<< HEAD
línea de la rama actual (main)
=======
línea de la rama que fusionas
>>>>>>> nova-rama
```

▶ Pasos: ver ficheros en conflicto → editar y dejar el texto final (borrando <<<, ===, >>>) → add → commit

```
git status
nano fitxer.txt
git add fitxer.txt
git commit -m "Resol conflicte"
```

▶ Atajo: quedarte con TU versión o con la de la otra rama (luego git add + commit)

```
git checkout --ours fitxer.txt
git checkout --theirs fitxer.txt
```

5.5 Metodologías de ramas (Git Flow)

Rama	Para qué sirve
master / main	Código estable en producción. Cada commit importante lleva una etiqueta de versión.
develop	Integración del desarrollo en curso.
feature/*	Una funcionalidad concreta. Sale de develop y vuelve a develop.
release/*	Preparación de una versión (últimos ajustes). Acaba en main y develop.
hotfix/*	Arreglo urgente en producción. Sale de main y se fusiona en main y develop.

6. Etiquetas (tags)

Marcan un punto concreto del historial, normalmente una versión. Hay dos tipos: **ligeras** (solo un nombre) y **anotadas** (con mensaje, autor y fecha; son las recomendadas).

▶ Listar etiquetas / filtrar por patrón / ver detalle de una

```
git tag
git tag -l 'v1.*'
git show v1.0
```

▶ Crear etiqueta ligera / anotada (sobre el commit actual)

```
git tag v1.0
git tag -a v1.0 -m "Versió 1.0"
```

▶ Etiquetado tardío: etiquetar un commit antiguo usando su hash

```
git log --oneline
git tag -a v0.9 -m "Versió 0.9" abc1234
```

▶ Subir una etiqueta / todas (git push normal NO sube las etiquetas)

```
git push origin v1.0
git push origin --tags
```

▶ Borrar en local / borrar en remoto (las dos últimas líneas son equivalentes)

```
git tag -d v1.0
git push origin :refs/tags/v1.0
git push origin --delete v1.0
```

▶ Obtener el código de una etiqueta: se crea una rama nueva que parte de ella

```
git checkout -b rama-desde-v1 v1.0
```

7. .gitignore

Fichero en la raíz del repo con patrones de lo que Git debe **ignorar** (logs, dependencias, secretos, binarios...). El propio `.gitignore` sí se versiona.

▶ Ignorar el directorio logs (ejercicio 3 del examen) y comprobar que ya no aparece

```
echo "logs/" > .gitignore
cat .gitignore
git status
```

▶ Ejemplo de .gitignore típico

```
logs/
*.log
*.tmp
.env
auth.txt
node_modules/
__pycache__/
*.pyc
.vscode/
.idea/
build/
dist/
.DS_Store
!importante.log
```

Patrón	Significado
`logs/`	Ignora cualquier directorio llamado logs (y su contenido).
`*.log`	Ignora todos los ficheros que acaban en .log.
`/build`	Solo el build de la raíz (la barra inicial ancla el patrón a la raíz).
`!importante.log`	Excepción: sí se versiona aunque coincida con otro patrón.

Patrón	Significado
`# comentario`	Línea de comentario.

▶ Si el fichero YA estaba versionado, ignorarlo no basta: hay que dejar de seguirlo. Y comprobar patrones

```
git rm -r --cached logs
git commit -m "Deixa de seguir logs"
git check-ignore -v logs/app.log
```

EXAMEN. Ejercicio 3: primero un commit **solo** con `.gitignore` (`git add .gitignore`), después otro commit con `script.py`. No uses `git add .`.

8. Deshacer cosas

Situación	Solución
Edité un fichero y quiero volver a la versión del último commit	`git restore fitxer`
Hice `git add` y me arrepiento (el cambio se conserva)	`git restore --staged fitxer`
Me equivoqué en el mensaje del último commit (sin push)	`git commit --amend -m "Nou"`
Quiero anular el último commit pero conservar los cambios	`git reset --soft HEAD~1`
Anular último commit y sacar los cambios de staging	`git reset HEAD~1`
Anular último commit y BORRAR los cambios	`git reset --hard HEAD~1`
Deshacer un commit ya publicado sin reescribir historia	`git revert abc1234`
Cambio de rama sin acabar y no quiero commitear	`git stash` … `git stash pop`

▶ Guardar cambios en el cajón / listar / recuperar y borrar / recuperar y conservar / borrar / historial de movimientos de HEAD (salvavidas)

```
git stash
git stash list
git stash pop
git stash apply
git stash drop
git reflog
```

▶ Crear un commit que deshace otro / tirar tu rama a lo que haya en remoto (pierdes lo local)

```
git revert abc1234
git revert HEAD
git reset --hard origin/main
```

OJO. `reset --hard` y `push --force` destruyen trabajo. En un repo compartido usa `revert`.

9. Colaborar: fork y pull request

9.1 Fork

Hacer fork es copiar un repositorio ajeno a tu cuenta para experimentar sin afectar al original. Después lo clonas y trabajas como con cualquier repo tuyo.

▶ Fork + clonado con gh (en la web: botón Fork arriba a la derecha)

```
gh repo fork OWNER/REPO --clone
cd REPO
git remote -v
```

▶ Enlazar el repo original como upstream y actualizar tu fork con sus novedades

```
git remote add upstream https://github.com/OWNER/REPO.git
git fetch upstream
git switch main
git merge upstream/main
git push origin main
```

> **NOTA.** El nombre `upstream` es solo una convención (se puede llamar como quieras). En los apuntes aparece `git merge upstream`; lo correcto es indicar la rama: `upstream/main` (o `upstream/master`).

9.2 Pull request (PR)

Tras hacer cambios en tu fork, propones al dueño del original que los incorpore mediante una pull request.

- Trabaja en una rama de tu fork y súbela: `git push -u origin mi-rama`.
- En GitHub abre el repo original → **New pull request** (o «Compare & pull request»).
- Elige **base** (repo original, rama destino) y **head** (tu fork, tu rama). Opción «compare across forks» si hace falta.
- Escribe título y descripción, pide revisores y pulsa **Create pull request**.
- El dueño revisa y hace **Merge pull request**; la PR pasa a estado **Merged**.

▶ Crear una PR desde terminal

```
git switch -c mi-millora
echo "canvi" >> Readme.md
git add Readme.md
git commit -m "Afegeix millora"
git push -u origin mi-millora
gh pr create --base main --title "Afegeix millora" --body "Descripció dels canvis"
```

▶ Listar / ver / probar en local / fusionar PRs / crear una issue

```
gh pr list
gh pr view 1
gh pr checkout 1
gh pr merge 1 --merge
gh issue create --title "Error" --body "Descripció"
```

10. Buenas prácticas con Git

10.1 Mensajes de commit

- Simples y claros. Separa **asunto** y **cuerpo** con una línea en blanco.
- Asunto: máximo 50 caracteres, empieza con mayúscula, **sin punto final**, en imperativo.
- Cuerpo: líneas de máximo 72 caracteres; explica **qué** se ha resuelto y **cómo**.

▶ Commit con asunto y cuerpo (dos -m)

```
git commit -m "Afegeix validació del formulari" \
  -m "El camp correu es valida amb una expressió regular."
```

> **EXAMEN.** En el examen los mensajes son **exactos** (por ejemplo `Primer commit`, `Afegeix pàgina principal`). Cópialos tal cual: mayúsculas, acentos y espacios incluidos.

10.2 Reglas de oro

- **No uses** `git add .` a ciegas: añade solo los ficheros con los que has trabajado (evita basura y ficheros secretos).
- No subas commits con código a medias: comparte código completo.
- Haz commits **por funcionalidad**: así se pueden revertir sin problemas. Si hay cambios sin relación, van en otro commit.
- Acuerda una metodología de ramas (Git Flow u otra) y cuándo crear ramas nuevas.
- Haz `git pull` antes de empezar a trabajar y `git status` antes de cada commit.
- Usa `.gitignore` y no subas contraseñas ni tokens.

11. GitHub Pages

Publica una web estática (ficheros `.html`, o un `README.md` renderizado) directamente desde un repositorio de GitHub. Los repos deben ser **públicos** (en cuentas gratuitas).

Tipo de sitio	Nombre del repo	URL resultante
Sitio de usuario	`USUARI.github.io`	`https://USUARI.github.io/`
Sitio de proyecto	cualquiera (ej. `exercici-2`)	`https://USUARI.github.io/exercici-2/`

11.1 Activarlo desde la web

- Entra en el repo → **Settings** → **Pages** (menú izquierdo).
- En **Build and deployment** → **Source**: *Deploy from a branch*.
- En **Branch** elige `main` y la carpeta `/ (root)` → **Save**.
- Espera 1–2 minutos. Arriba aparecerá «Your site is live at…».
- Opcional: **Theme chooser** (solo funciona si el contenido es un `README.md`).

11.2 Activarlo desde la terminal

▶ Activar Pages con la rama main y carpeta raíz

```
gh api -X POST /repos/$USUARI/$USUARI.github.io/pages \
  -f "source[branch]=main" -f "source[path]=/"
```

▶ Comprobar la URL y el estado / ver el despliegue en Actions (workflow «pages build and deployment»)

```
gh api /repos/$USUARI/$USUARI.github.io/pages --jq '.html_url, .status'
gh run list --limit 5
```

11.3 Receta: repo <usuari>.github.io con web

▶ Si ya existía: borrarlo y crearlo de nuevo (si falla por permisos: Settings → Danger Zone → Delete)

```
gh repo delete $USUARI/$USUARI.github.io --yes
gh repo create $USUARI.github.io --public
```

▶ Subir el CONTENIDO del zip a la raíz del repo (index.html debe quedar en la raíz, no en una subcarpeta)

```
cd ~
mkdir -p ex4 && cd ex4
unzip ../exercici-4.zip
ls -la
git init -b main
git remote add origin https://github.com/$USUARI/$USUARI.github.io.git
git add .
git commit -m "Primer commit"
git push -u origin main
```

▶ Editar README.md añadiendo tu nombre al final, commit y push (provoca un nuevo despliegue)

```
echo "$NOM" >> README.md
git add README.md
git commit -m "Afegeix el meu nom"
git push
```

> **OJO.** Si ves **404**: el fichero debe llamarse `index.html` (minúsculas) y estar en la raíz de la rama publicada; espera un par de minutos; comprueba que el repo es público y que Pages apunta a `main` y `/ (root)`.

▶ Web mínima para probar Pages (index.html)

```
<!DOCTYPE html>
<html lang="ca">
<head>
  <meta charset="utf-8">
  <title>La meua web</title>
</head>
<body>
  <h1>Hola món</h1>
</body>
</html>
```

12. CI/CD y GitHub Actions: conceptos

12.1 Qué es CI/CD

Concepto	Qué es
Continuous Integration (CI)	Los desarrolladores integran código en un repo compartido muchas veces al día. Cada integración se verifica con **tests automáticos** para detectar errores pronto.
Continuous Delivery (CD)	Extensión de CI: el código está **siempre listo para liberarse** a producción. Pasa pruebas y validaciones, pero el despliegue final lo decide una persona.
Continuous Deployment (CD)	Cada cambio aprobado y verificado se **despliega a producción automáticamente**, sin intervención humana. Exige mucha confianza en los tests.

Ciclo: Codificar → Build → Testear → Crear release → Desplegar. CI cubre hasta testear; Delivery llega hasta release; Deployment llega hasta desplegar.

12.2 Componentes de GitHub Actions

Componente	Descripción
Workflow	Procedimiento automatizado definido en un fichero YAML del repo. Tiene uno o más jobs y se activa por un **evento** (push, pull request…).
Job	Conjunto de steps que se ejecutan en el **mismo runner**. Por defecto los jobs corren **en paralelo** (se puede cambiar con `needs`).
Step	Tarea individual dentro de un job: una **action** (`uses`) o un comando de shell (`run`). Se ejecutan en secuencia.
Action	Bloque reutilizable más pequeño. Puede ser de GitHub, de terceros (Marketplace) o propia.
Runner	Servidor que ejecuta los jobs y devuelve el resultado.

Jerarquía: **Workflow → Job → Step → Action**.

12.3 Runners

	GitHub-hosted	Self-hosted
Dónde	Máquinas en la nube de GitHub	Tu propia máquina / servidor
Sistemas	Ubuntu (Linux), Windows y macOS	Los que tú instales
Mantenimiento	Ninguno (todo configurado)	Lo configuras y mantienes tú
Coste	Gratis con límite (2000 min/mes en plan gratuito), luego de pago	No pagas por su uso
Flexibilidad	Limitada en configuraciones muy específicas	Total; acceso a tu red local

Etiqueta runs-on	Sistema
`ubuntu-latest`	Linux (Ubuntu)
`windows-latest`	Windows
`macos-latest`	macOS
`self-hosted`	Tu runner propio

13. Workflows: sintaxis y ejemplos listos para copiar

Los workflows son ficheros **YAML** en el directorio `.github/workflows/` en la raíz del repo (cualquier nombre acabado en `.yml` o `.yaml`). Cada push de ese fichero activa su ejecución si coincide el evento. Ver resultados: pestaña **Actions** del repo.

13.1 Primer workflow y anatomía

▶ Workflow mínimo (ejemplo de los apuntes)

```
name: primer-ejemplo
on: [push]
jobs:
  check-bats-version:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-node@v4
      - run: npm install -g bats
      - run: bats -v
```

Clave	Función
`name`	(Opcional) Nombre del workflow que sale en la pestaña Actions.
`on`	**Requerido.** Evento(s) que lanzan el workflow.
`jobs`	Agrupa los jobs. Cada job necesita un identificador (`check-bats-version`).

Clave	Función
`jobs.<id>.runs-on`	**Requerido.** Máquina donde corre el job.
`jobs.<id>.needs`	Jobs que deben acabar antes (evita ejecución en paralelo).
`jobs.<id>.steps`	Lista de pasos secuenciales.
`steps.uses`	Ejecuta una action (`actions/checkout@v6`).
`steps.run`	Ejecuta un comando de shell.
`steps.with`	Parámetros de entrada de la action.
`env`	Variables de entorno (workflow, job o step; gana la más específica).
`strategy.matrix`	Genera varios jobs cambiando variables.

13.2 Crear y subir el workflow

▶ El directorio debe llamarse exactamente .github/workflows (con el punto)

```
mkdir -p .github/workflows
nano .github/workflows/tests.yml
git add .github/workflows/tests.yml
git commit -m "Afegeix workflow de tests"
git push
```

▶ Ver ejecuciones desde terminal: lista / seguir en vivo / logs / solo logs fallidos / workflows / lanzar uno manual (requiere workflow_dispatch)

```
gh run list
gh run watch
gh run view --log
gh run view --log-failed
gh workflow list
gh workflow run tests.yml
```

13.3 Eventos (on)

▶ Solo en push

```
on: [push]
```

▶ En push y en pull request

```
on: [push, pull_request]
```

▶ Solo para la rama main

```
on:
  push:
    branches:
      - main
  pull_request:
    branches:
      - main
```

▶ Con filtro de ramas, etiquetas y rutas

```
on:
  push:
    branches: [main]
    tags:
      - v1
    paths:
      - "src/**"
```

▶ Lanzamiento manual desde la web + programado (lunes a las 08:00 UTC)

```
on:
  workflow_dispatch:
  schedule:
    - cron: "0 8 * * 1"
```

13.4 Ejemplo del examen: tests unitarios en Windows, Linux y Mac

Tres jobs (`test-win`, `test-linux`, `test-mac`) que ejecutan `python -m unittest -v` en cada push a `main`. Windows con Python 3.12 y Linux/Mac con 3.13, usando `actions/checkout@v6` y `actions/setup-python@v6`.

▶ .github/workflows/tests.yml (versión del examen, copiar y pegar)

```
name: Tests
on:
  push:
    branches: [main]
jobs:
  test-win:
    runs-on: windows-latest
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-python@v6
        with:
          python-version: "3.12"
      - run: python -m unittest -v

  test-linux:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-python@v6
        with:
          python-version: "3.13"
      - run: python -m unittest -v

  test-mac:
    runs-on: macos-latest
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-python@v6
        with:
          python-version: "3.13"
      - run: python -m unittest -v
```

EXAMEN. Pon `python-version` **entre comillas**: `3.13` sin comillas funciona, pero `3.10` se interpretaría como el número 3.1. Los dos scripts de Python (programa y tests) deben estar en la **raíz del repo** para que `unittest` los encuentre; si están en una subcarpeta, añade `working-directory: carpeta` al step `run`.

▶ Flujo completo del ejercicio 5 (primero crea el repo vacío: gh repo create exercici-5 --public)

```
cd ~
mkdir -p ex5 && cd ex5
unzip ../exercici-5.zip
git init -b main
git remote add origin https://github.com/$USUARI/exercici-5.git
mkdir -p .github/workflows
nano .github/workflows/tests.yml
python3 -m unittest -v
git add .
git commit -m "Afegeix tests i workflow"
git push -u origin main
gh run watch
```

13.5 Lo mismo con matrix (más corto)

▶ Genera 3 x 2 = 6 jobs a partir de una sola definición

```
name: Tests matrix
on:
  push:
    branches: [main]
jobs:
  test:
    runs-on: ${{ matrix.os }}
    strategy:
      matrix:
        os: [ubuntu-latest, windows-latest, macos-latest]
        python-version: ["3.12", "3.13"]
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-python@v6
        with:
          python-version: ${{ matrix.python-version }}
      - run: python -m unittest -v
```

> **OJO.** Ojo: el examen pide **3 jobs con esos nombres exactos** (`test-win`, `test-linux`, `test-mac`); una matrix crearía otros nombres. Usa la versión larga del apartado 13.4.

▶ matrix con include (variables asociadas a cada combinación) y fail-fast: false

```
strategy:
  matrix:
    include:
      - node-version: 18.x
        site: "prod"
      - node-version: 20.x
        site: "dev"
  fail-fast: false
steps:
  - env:
      SITE: ${{ matrix.site }}
    run: echo $SITE
```

13.6 Dependencias entre jobs (needs) y condiciones (if)

▶ Por defecto los jobs van en paralelo; needs los encadena; if: always() ignora los fallos previos

```
jobs:
  job1:
    runs-on: ubuntu-latest
    steps:
      - run: echo "job1"
  job2:
    needs: job1
    runs-on: ubuntu-latest
    steps:
      - run: echo "job2 se ejecuta cuando job1 acaba bien"
  job3:
    needs: [job1, job2]
    if: always()
    runs-on: ubuntu-latest
    steps:
      - run: echo "job3 se ejecuta siempre, falle o no job1/job2"
```

13.7 Variables de entorno, run multilínea y contextos

▶ env a nivel workflow/job/step, run con varios comandos (|) y working-directory

```
name: Variables
on: push
env:
  SERVER: production
jobs:
  my-job:
    runs-on: ubuntu-latest
    env:
      FIRST_NAME: Mona
    steps:
      - name: Saludo
        env:
          MY_VAR: Hola, em dic
        run: |
          echo $MY_VAR $FIRST_NAME
          echo "Servidor: $SERVER"
          echo "Usuari: ${{ github.actor }} en ${{ runner.os }}"
      - name: Limpiar carpeta temporal
        run: rm -rf *
        working-directory: ./temp
```

Expresión	Valor
`${{ github.actor }}`	Usuario que lanzó el workflow
`${{ github.repository }}`	owner/repo
`${{ github.ref_name }}`	Nombre de la rama o etiqueta
`${{ runner.os }}`	Linux, Windows o macOS
`${{ secrets.NOM }}`	Secreto del repo (Settings → Secrets and variables → Actions)
`${{ matrix.x }}`	Valor de la matrix en esa ejecución
`${{ steps.id.outputs.x }}`	Salida de un step anterior

13.8 Otros ejemplos útiles

▶ CI de un proyecto Node

```
name: Node CI
on: [push, pull_request]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-node@v4
        with:
          node-version: 20
      - run: npm ci
      - run: npm test
```

▶ Publicar en Pages con Actions (en Settings → Pages elige Source: GitHub Actions). Revisa las versiones de cada action en el Marketplace

```
name: Deploy Pages
on:
  push:
    branches: [main]
permissions:
  contents: read
  pages: write
  id-token: write
jobs:
  deploy:
    runs-on: ubuntu-latest
    environment:
      name: github-pages
      url: ${{ steps.deployment.outputs.page_url }}
    steps:
      - uses: actions/checkout@v6
      - uses: actions/configure-pages@v5
      - uses: actions/upload-pages-artifact@v4
        with:
          path: .
      - id: deployment
        uses: actions/deploy-pages@v4
```

▶ Guardar un fichero generado como artefacto descargable

```
steps:
  - uses: actions/checkout@v6
  - run: echo "hola" > resultat.txt
  - uses: actions/upload-artifact@v4
    with:
      name: resultat
      path: resultat.txt
```

OJO. Errores típicos de YAML: usar **tabuladores** (solo espacios, 2 por nivel), mala indentación, olvidar el `-` en los steps, y que el directorio no sea exactamente `.github/workflows`.

14. Custom GitHub Actions (acciones propias)

Son tareas individuales que combinas dentro de un step de un job de un workflow. Se definen con un fichero de **metadatos** `action.yml` (o `.yaml`).

14.1 Tipos

Tipo	Ventajas	Inconvenientes
Docker container	Más consistente y fiable; no te preocupas por dependencias.	Solo runners **Linux**; más lentas.
JavaScript	Código más simple y ejecución más rápida; corre en todos los runners.	Debe ser JavaScript puro para ser portable.
Composite (pasos compuestos)	Agrupa varios steps (run/uses) en una sola action.	Menos potencia que código propio.

14.2 El fichero action.yml

Clave	Obligatoria	Para qué sirve
`name`	Sí	Nombre de la Action que GitHub muestra en la pestaña Actions.
`author`	No	Autor de la Action.
`description`	Sí	Descripción breve.
`inputs`	No	Parámetros de entrada. Todos sus atributos son requeridos salvo `default`.
`outputs`	No	Datos que la Action establece. Todos sus atributos son requeridos.
`runs`	Sí	Cómo se ejecuta; cambia según sea JavaScript, Docker o composite.
`branding`	No	Color e icono (feathericons.com) si se publica en el Marketplace.

> **NOTA.** Por cada input GitHub crea una variable de entorno `INPUT_<NOMBRE>`: el nombre se pasa a **mayúsculas** y los espacios se reemplazan por `_`.

runs según el tipo

▶ JavaScript: using y main son requeridos; pre/post son opcionales (pre-if / post-if para condicionarlos). En los apuntes pone node12, ya obsoleto: usa node20 o superior

```
runs:
  using: "node20"
  pre: "setup.js"
  main: "index.js"
  post: "cleanup.js"
```

▶ Docker: using e image requeridos (Dockerfile local o imagen pública); args = entradas del contenedor; entrypoint invalida el ENTRYPOINT del Dockerfile

```
runs:
  using: "docker"
  image: "Dockerfile"
  args:
    - ${{ inputs.who-to-greet }}
  entrypoint: "main.sh"
  post-entrypoint: "cleanup.sh"
```

14.3 Ejemplo JavaScript paso a paso

Pasos: crear `action.yml` → crear proyecto Node → instalar librerías → `index.js` → compilar → subir y probar en un workflow.

▶ Crear el directorio de la action, proyecto Node y dependencias (@actions/core y @actions/github)

```
mkdir -p .github/actions/hello-world-javascript-action
cd .github/actions/hello-world-javascript-action
npm init -y
npm install @actions/core @actions/github
npm install -g @vercel/ncc
```

▶ .github/actions/hello-world-javascript-action/action.yml

```
name: 'Hello World'
description: 'Greet someone and record the time'
inputs:
  who-to-greet:
    description: 'Who to greet'
    required: true
    default: 'World'
outputs:
  time:
    description: 'The time we greeted you'
runs:
  using: 'node20'
  main: 'dist/index.js'
```

▶ index.js (lógica principal: leer input, escribir output, mostrar el payload del evento)

```
const core = require('@actions/core');
const github = require('@actions/github');

try {
  const nameToGreet = core.getInput('who-to-greet');
  console.log('Hello ' + nameToGreet + '!');
  const time = (new Date()).toTimeString();
  core.setOutput('time', time);
  const payload = JSON.stringify(github.context.payload, undefined, 2);
  console.log('The event payload: ' + payload);
} catch (error) {
  core.setFailed(error.message);
}
```

▶ Compilar a dist/index.js: así no hace falta subir node_modules al repo (main apunta a dist/index.js)

```
ncc build index.js --license licenses.txt
```

▶ Probar la action en un workflow (checkout obligatorio para poder usar una action local)

```
on: [push]
jobs:
  hello_world_job:
    runs-on: ubuntu-latest
    name: A job to say hello
    steps:
      - name: Checkout
        uses: actions/checkout@v6
      - name: Hello world action step
        id: hello
        uses: ./.github/actions/hello-world-javascript-action
        with:
          who-to-greet: 'Mona the Octocat'
      - name: Get the output time
        run: echo "The time was ${{ steps.hello.outputs.time }}"
```

14.4 Ejemplo Docker paso a paso

Pasos: crear Dockerfile (+ script) → crear `action.yml` → subir y probar en un workflow.

▶ Dockerfile: imagen base, copia del script y ENTRYPOINT

```
FROM alpine:3.20
COPY entrypoint.sh /entrypoint.sh
RUN chmod +x /entrypoint.sh
ENTRYPOINT ["/entrypoint.sh"]
```

▶ entrypoint.sh ($1 = primer arg). En los apuntes aparece ::set-output, ya obsoleto: se usa $GITHUB_OUTPUT

```
#!/bin/sh -l

echo "Hello $1"
time=$(date)
echo "time=$time" >> $GITHUB_OUTPUT
```

▶ action.yml en la raíz del repo (el workflow la usa con uses: ./)

```
name: 'Hello World'
description: 'Greet someone and record the time'
inputs:
  who-to-greet:
    description: 'Who to greet'
    required: true
    default: 'World'
outputs:
  time:
    description: 'The time we greeted you'
runs:
  using: 'docker'
  image: 'Dockerfile'
  args:
    - ${{ inputs.who-to-greet }}
```

▶ Workflow de prueba (uses: ./ = action en el directorio raíz)

```
on: [push]
jobs:
  hello_world_job:
    runs-on: ubuntu-latest
    name: A job to say hello
    steps:
      - name: Checkout
        uses: actions/checkout@v6
      - name: Hello world action step
        uses: ./
        id: hello
        with:
          who-to-greet: 'Mona the Octocat'
      - name: Get the output time
        run: echo "The time was ${{ steps.hello.outputs.time }}"
```

14.5 Ejemplo composite (pasos compuestos)

▶ En una action composite cada step run necesita shell: bash (o pwsh, etc.)

```
name: 'Saluda'
description: 'Action compuesta'
inputs:
  nom:
    description: 'Nom'
    required: true
    default: 'Món'
runs:
  using: 'composite'
  steps:
    - run: echo "Hola ${{ inputs.nom }}"
      shell: bash
```

15. Examen resuelto paso a paso (copiar y pegar)

Resolución del examen práctico «Git, GitHub Pages y GitHub Actions» que viene en tus documentos. Si el examen real cambia nombres o mensajes, **respeta siempre lo que diga el enunciado**: nombres de repo, de fichero y mensajes de commit exactos.

> **EXAMEN.** Todos los repos **públicos** y en la rama **main**. Lo que se corrige es el historial del repo y la pestaña Actions. Si un ejercicio se bloquea, pasa al siguiente.

Paso 0. Preparación (no puntúa)

▶ Identidad, token (con todos los permisos marcados), login y variables

```
git config --global user.name "Nom Cognom"
git config --global user.email "correu@gva.es"
git config --global init.defaultBranch main
cat > ~/auth.txt
# pega el token, Enter, Ctrl+D
gh auth login --with-token < ~/auth.txt
gh auth setup-git
gh auth status
USUARI=$(gh api user -q .login)
NOM="Nom Cognom"
```

Ejercicio 1: Repo (2 puntos)

▶ Los 9 pasos del ejercicio 1 (el último push asegura que el tercer commit llega a GitHub)

```
gh repo create exercici-1 --public
mkdir -p ~/examen && cd ~/examen
git init -b main
git remote add origin https://github.com/$USUARI/exercici-1.git
echo "# Respostes — $NOM" > respostes.md
git add respostes.md
git commit -m "Primer commit"
git push -u origin main
echo "$NOM" > Readme.md
git add Readme.md
git commit -m "Segon commit"
git log --oneline > log.txt
git add log.txt
git commit -m "Tercer commit"
git push
```

> **OJO.** `~/examen` pasa a ser el repo de exercici-1. Termina el ejercicio 1 **antes** del 2 y **no uses** `git add .` dentro de `~/examen` (el ejercicio 2 crea una carpeta con su propio repo dentro).

Ejercicio 2: Repo (2 puntos)

▶ Ejercicio 2 completo. Dos commits, cada uno con UN solo fichero, y push final

```
gh repo create exercici-2 --public
mkdir -p ~/examen/exercici-2 && cd ~/examen/exercici-2
git init -b main
git remote add origin https://github.com/$USUARI/exercici-2.git
cat > index.html << EOF
<!DOCTYPE html>
<html lang="ca">
<head>
 <meta charset="utf-8">
 <title>Portfoli de $NOM</title>
 <link rel="stylesheet" href="estil.css">
</head>
<body>
 <h1>Portfoli de $NOM</h1>
 <p>Pàgina publicada amb GitHub Pages.</p>
</body>
</html>
EOF
git add index.html
git commit -m "Afegeix pàgina principal"
echo "# Capçalera" > Readme.md
git add Readme.md
git commit -m "Afegeix readme"
git push -u origin main
```

TRUCO. Comprueba `cat index.html`: el nombre y apellido deben haberse sustituido en `<title>` y en `<h1>`. Si no usas la variable, escribe tu nombre a mano.

Ejercicio 3: .gitignore (1 punto)

Descarga `exercici-3.zip` de https://github.com/titansmc/examen-daw (en el navegador: abre el fichero y pulsa el botón de descarga). Por terminal, alguna de estas opciones:

▶ Opción A: clonar el repo del profesor y copiar el zip (comprueba su contenido con unzip -l)

```
cd ~
gh repo clone titansmc/examen-daw
cp examen-daw/exercici-3.zip ~/
unzip -l ~/exercici-3.zip
```

▶ Opción B: descarga directa (solo si la rama es main y el zip está en la raíz del repo)

```
cd ~
curl -L -o exercici-3.zip https://github.com/titansmc/examen-daw/raw/main/exercici-3.zip
```

▶ Descomprimir, entrar, ignorar logs, repo, primer commit SOLO con .gitignore, segundo con script.py

```
cd ~
unzip exercici-3.zip
cd exercici-3
ls -la
echo "logs/" > .gitignore
gh repo create exercici-3 --public
git init -b main
git remote add origin https://github.com/$USUARI/exercici-3.git
git add .gitignore
git commit -m "Afegeix .gitignore"
git push -u origin main
git add script.py
git commit -m "Afegeix script"
git push
git status
```

NOTA. Si `unzip` extrae los ficheros sueltos (sin carpeta `exercici-3`), haz antes `mkdir ~/exercici-3 && unzip ~/exercici-3.zip -d ~/exercici-3`. `git status` final no debe mostrar `logs/`.

Ejercicio 4: GitHub Pages (2 puntos)

▶ Si ya existía el repo <usuari>.github.io: borrarlo y crearlo (si falla, bórralo en Settings → Danger Zone)

```
cd ~
gh repo delete $USUARI/$USUARI.github.io --yes
gh repo create $USUARI.github.io --public
```

Descarga `exercici-4.zip` igual que el 3 (cambia el nombre del zip) y sube su **contenido**:

▶ Subir el contenido y activar Pages (main, / (root)). Mientras se publica, ve al ejercicio 5

```
mkdir -p ~/ex4 && cd ~/ex4
unzip ~/exercici-4.zip
ls -la
git init -b main
git remote add origin https://github.com/$USUARI/$USUARI.github.io.git
git add .
git commit -m "Primer commit"
git push -u origin main
gh api -X POST /repos/$USUARI/$USUARI.github.io/pages \
  -f "source[branch]=main" -f "source[path]=/"
```

▶ Cuando la página esté disponible: añadir tu nombre al final de README.md, commit y push

```
cd ~/ex4
echo "$NOM" >> README.md
git add README.md
git commit -m "Afegeix el meu nom"
git push
gh api /repos/$USUARI/$USUARI.github.io/pages --jq '.html_url, .status'
```

> **OJO.** Si el zip se descomprime dentro de una carpeta, **mueve el contenido a la raíz** del repo (`mv carpeta/* .`) para que `index.html` quede en la raíz.

Ejercicio 5: GitHub Actions (3 puntos)

▶ Crear el repo, descomprimir y probar los tests en local (los .py deben quedar en la raíz del repo)

```
gh repo create exercici-5 --public
mkdir -p ~/ex5 && cd ~/ex5
unzip ~/exercici-5.zip
ls -la
python3 -m unittest -v
```

▶ Crear el workflow con los 3 jobs (heredoc entre comillas 'EOF' para que no toque las ${{ }})

```
mkdir -p .github/workflows
cat > .github/workflows/tests.yml << 'EOF'
name: Tests
on:
  push:
    branches: [main]
jobs:
  test-win:
    runs-on: windows-latest
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-python@v6
        with:
          python-version: "3.12"
      - run: python -m unittest -v

  test-linux:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-python@v6
        with:
          python-version: "3.13"
      - run: python -m unittest -v

  test-mac:
    runs-on: macos-latest
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-python@v6
        with:
          python-version: "3.13"
      - run: python -m unittest -v
EOF
```

▶ Subir todo y ver la ejecución (los tres jobs deben quedar en verde en la pestaña Actions)

```
git init -b main
git remote add origin https://github.com/$USUARI/exercici-5.git
git add .
git commit -m "Afegeix tests i workflow"
git push -u origin main
gh run watch
```

OJO. Si el push se rechaza por «workflow scope», tu token no tiene marcado el permiso **workflow**: genera otro con todos los permisos y repite `gh auth login --with-token`.

Comprobación final de todo

▶ Repos públicos, mensajes de commit de cada repo y estado de las ejecuciones de Actions

```
gh repo list --limit 20 --json name,visibility
for r in exercici-1 exercici-2 exercici-3 exercici-5 $USUARI.github.io; do
  echo "== $r"
  gh api repos/$USUARI/$r/commits --jq '.[].commit.message'
done
gh run list --repo $USUARI/exercici-5
```

16. Preguntas cortas de razonamiento (con respuesta)

El examen incluye dos preguntas cortas de razonamiento. Respuestas modelo, para adaptar con tus palabras:

Pregunta	Respuesta corta
¿Diferencia entre `git fetch` y `git pull`?	`fetch` solo descarga los cambios del remoto a una rama oculta (origin/main) sin tocar tu trabajo. `pull` hace fetch y además los fusiona (merge) en tu rama actual.
¿Qué hace `git push -u origin main`?	Sube la rama main a origin y asocia ambas ramas, de modo que los siguientes `git push`/`git pull` no necesitan indicar remoto ni rama.
¿Por qué no es buena práctica `git add .`?	Añade todo lo que haya, incluidos ficheros basura, temporales o secretos, e infla el repo. Es mejor añadir solo los ficheros trabajados, y commits por funcionalidad.
¿Qué es un fork y en qué se diferencia de un clone?	El fork copia el repo a tu cuenta de GitHub (copia remota que puedes modificar sin afectar al original). El clone descarga un repo remoto a tu máquina.
¿Para qué sirve una pull request?	Para proponer al dueño del repo original que incorpore los cambios de tu rama/fork, con espacio para revisión y comentarios antes de hacer merge.
¿Diferencia entre `merge` y `rebase`?	Merge une dos ramas con un commit de fusión y conserva el historial real. Rebase reaplica tus commits sobre otra base y deja un historial lineal (reescribe commits).
¿Para qué sirve `.gitignore`?	Para indicar ficheros/directorios que Git no debe seguir (logs, dependencias, claves, binarios).
Etiqueta ligera vs anotada	Ligera: solo un nombre que apunta a un commit. Anotada (`-a -m`): objeto con mensaje, autor y fecha; recomendada para versiones.
`git revert` vs `git reset`	Revert crea un commit nuevo que deshace otro (seguro en repos compartidos). Reset mueve la rama hacia atrás y reescribe el historial.
¿Qué es CI y qué aporta?	Integrar código con frecuencia y ejecutar tests automáticos en cada integración para detectar errores pronto.
Delivery vs Deployment	Delivery deja el código siempre listo para producción, pero el despliegue es manual. Deployment lo despliega automáticamente en cuanto pasa las pruebas.
¿Por qué varios jobs en el examen?	Cada job corre en su propio runner; así se prueban los tests en Windows, Linux y macOS a la vez (por defecto en paralelo).
Runner GitHub-hosted vs self-hosted	Hosted: máquinas de GitHub sin mantenimiento (Linux, Windows, macOS), con límite de minutos gratis. Self-hosted: tu propia máquina, sin coste por uso pero la mantienes tú.
Action Docker vs JavaScript	Docker: consistente pero solo Linux y más lenta. JavaScript: más rápida y funciona en todos los runners.
¿Qué publica GitHub Pages?	Una web estática desde una rama (y carpeta) del repo: ficheros HTML o un README.md, accesible en usuario.github.io.
¿Qué hace `needs` en un job?	Indica que ese job espera a que terminen (bien) los jobs indicados, evitando la ejecución en paralelo.

17. Errores típicos y cómo arreglarlos

Error / síntoma	Causa y solución
fatal: not a git repository	No estás en un repo. `cd` a la carpeta correcta o `git init -b main`.
error: remote origin already exists	`git remote set-url origin URL`.
! [rejected] main -> main (fetch first / non-fast-forward)	El remoto tiene commits que no tienes (p. ej. README creado en la web). `git pull --rebase origin main` y luego `git push`.
fatal: refusing to merge unrelated histories	`git pull origin main --allow-unrelated-histories` y luego `git push`.
error: src refspec main does not match any	No hay commits o la rama se llama master. Haz un commit y `git branch -M main`.
fatal: The current branch main has no upstream branch	`git push -u origin main`.
Authentication failed / could not read Username	`gh auth login --with-token < ~/auth.txt` y `gh auth setup-git`. Revisa los permisos del token.
remote: Permission denied / 403	El repo no es tuyo o el token no tiene permisos. Revisa la URL (`git remote -v`) y los scopes.
refusing to allow a Personal Access Token to create or update workflow	Falta el permiso **workflow** en el token. Genera uno con todos los permisos.
nothing to commit, working tree clean	No hiciste `git add` o el fichero está ignorado. `git status`.
warning: adding embedded git repository	Hiciste `git add .` sobre una carpeta que contiene otro repo (~/examen / exercici-2). Añade ficheros uno a uno.
Commit con el mensaje mal	Sin push: `git commit --amend -m "Missatge"`. Con push: commit nuevo.
Se me coló un fichero en el commit	`git rm --cached fitxer`, añadirlo a `.gitignore` y `git commit --amend`.
Estoy en la rama equivocada	`git switch main` (si hay cambios sin commit: `git stash` antes, `git stash pop` después).
CONFLICT (content): Merge conflict in ...	Edita el fichero, borra los marcadores, `git add` y `git commit`. O cancela con `git merge --abort`.
Pages muestra 404	index.html en minúsculas en la raíz, Pages apuntando a main / (root), repo público, esperar 1–2 minutos.
El workflow no se ejecuta	Ruta exacta `.github/workflows/*.yml`, evento/rama correctos en `on`, YAML bien indentado (sin tabuladores). Revisa la pestaña Actions.
Tests fallan solo en Actions	Los .py no están en la raíz, falta `working-directory`, o la versión de Python es distinta.
warning: LF will be replaced by CRLF	Aviso inofensivo de saltos de línea.

18. Chuleta final de comandos

Git

Acción	Comando
Crear repo / clonar	`git init -b main` · `git clone URL`
Estado / diferencias	`git status` · `git diff` · `git diff --staged`
Añadir / commit	`git add fitxer` · `git commit -m "msg"` · `git commit -am "msg"`
Historial	`git log --oneline` · `git log --oneline --graph --all` · `git show hash`
Remotos	`git remote add origin URL` · `git remote -v` · `git remote set-url origin URL`
Subir / bajar	`git push -u origin main` · `git push` · `git pull` · `git fetch origin`
Ramas	`git branch` · `git switch -c rama` · `git switch rama` · `git branch -d rama`
Fusionar	`git switch main` · `git merge rama` · `git merge --abort`
Etiquetas	`git tag -a v1.0 -m "msg"` · `git push origin --tags` · `git tag -d v1.0`
Deshacer	`git restore f` · `git restore --staged f` · `git revert hash` · `git reset --soft HEAD~1`
Guardar a un lado	`git stash` · `git stash pop`
Renombrar rama a main	`git branch -M main`

GitHub CLI (gh)

Acción	Comando
Login	`gh auth login --with-token < auth.txt` · `gh auth setup-git` · `gh auth status`
Mi usuario	`gh api user -q .login`
Repos	`gh repo create NOM --public` · `gh repo list` · `gh repo clone USER/REPO` · `gh repo fork`
Pull requests	`gh pr create` · `gh pr list` · `gh pr merge N`
Actions	`gh run list` · `gh run watch` · `gh run view --log-failed` · `gh workflow run f.yml`
Pages	`gh api -X POST /repos/U/R/pages -f "source[branch]=main" -f "source[path]=/"`

Mensajes exactos del examen

Ejercicio	Mensajes de commit
1 · exercici-1	`Primer commit` → `Segon commit` → `Tercer commit`
2 · exercici-2	`Afegeix pàgina principal` → `Afegeix readme`
3 · exercici-3	`Afegeix .gitignore` → `Afegeix script`
4 · <usuari>.github.io	`Afegeix el meu nom`
5 · exercici-5	Workflow con jobs `test-win`, `test-linux`, `test-mac` (Python 3.12 / 3.13 / 3.13)